Project 12 — GAN Image Generation

Level: Advanced | **Dataset:** Fashion-MNIST (torchvision) | **Framework:** PyTorch

---

Objective

Build a DCGAN (Deep Convolutional GAN) to generate realistic fashion item images.
Cover: generator, discriminator, adversarial training loop, mode collapse, training stability tricks, FID score concept.

Project Structure

```
12_gan_fashion/
├── notebooks/
|   ├── 01_gan_theory.ipynb
|   ├── 02_train.ipynb
|   └── 03_evaluation.ipynb
├── data/
├── models/
|   ├── generator.pkl
|   └── discriminator.pkl
├── charts/
|   └── generated_grids/   (saved per epoch)
├── path_utils.py
├── dashboard_core.py
└── app.py
```

Notebook 01 — GAN Theory (`01_gan_theory.ipynb`)

STOP 1 — GAN Core Concept

Write theory cells:

- Two networks: Generator G and Discriminator D
- G: random noise z ~ N(0,1) → fake image
- D: real or fake image → probability of being real (scalar)
- Training: G tries to fool D, D tries to catch G
- **Agent stops here. Explain:**
  - The minimax game: min_G max_D [log D(x) + log(1 - D(G(z)))]
  - In English: D maximizes its ability to distinguish real/fake; G minimizes D's ability to do so
  - What "equilibrium" means: G generates images indistinguishable from real
  - Why this is harder to train than VAE (two losses, adversarial dynamics)
  - Real-world GAN applications: deepfakes, data augmentation, style transfer
- Wait for user confirmation before continuing

STOP 2 — GAN Training Dynamics

Write cells explaining:

- D training step: maximize log D(x_real) + log(1 - D(G(z))) → D loss = BCE on real=1 + fake=0
- G training step: maximize log D(G(z)) [non-saturating] → G loss = BCE on fake=1 (trick D to think fake is real)
- Why non-saturating G loss: early training D easily rejects fakes → log(1-D(G(z))) saturates to 0
- **Agent stops here. Explain:**
  - The original saturating loss problem: D(G(z)) ≈ 0 early → gradient of log(1-0)=0 → G gets no gradient
  - Non-saturating fix: flip labels for G → maximize log D(G(z)) instead of minimize log(1-D(G(z)))
  - Practical effect: G gets strong gradients even when it's bad at fooling D
  - This is the standard trick used in all modern GAN implementations
- Wait for confirmation

STOP 3 — Mode Collapse

Write cells with visual examples:

- Mode collapse: G learns to generate one type of output (one mode) that fools D
- Example: only generates sneakers even for Fashion-MNIST with 10 categories
- How to detect: generated samples all look identical, lack diversity
- **Agent stops here. Explain:**
  - Why mode collapse happens: G finds a "shortcut" — one image that reliably fools D
  - D then adapts to reject that image → G finds another mode → cycling
  - Solutions: minibatch discrimination, spectral normalization, Wasserstein distance (WGAN)
  - How to monitor: visually inspect generated grids each epoch
- Wait for confirmation

Notebook 02 — Training (`02_train.ipynb`)

STOP 4 — Generator Architecture (DCGAN)

```
Input: z ~ N(0,1), shape [B, 100]
Linear(100, 7*7*256) → reshape to [B, 256, 7, 7]
ConvTranspose2d(256, 128, 4, 2, 1) → BN → ReLU → [B, 128, 14, 14]
ConvTranspose2d(128, 64, 4, 2, 1)  → BN → ReLU → [B, 64, 28, 28]
ConvTranspose2d(64, 1, 3, 1, 1)    → Tanh           → [B, 1, 28, 28]
```

- **Agent stops here. Explain:**
  - What ConvTranspose2d is: the reverse of Conv2d — upsamples spatial dimensions
  - How ConvTranspose2d with (kernel=4, stride=2, pad=1) doubles spatial size: 7→14→28
  - Why Tanh at output: output in [-1, 1] → normalize real images to [-1,1] (not [0,1])
  - Why we start with 7×7×256 dense: build up from tiny representation
  - What the noise z represents: different z = different image
- Wait for confirmation

STOP 5 — Discriminator Architecture (DCGAN)

```
Input: [B, 1, 28, 28]
Conv2d(1, 64, 4, 2, 1)   → LeakyReLU(0.2) → [B, 64, 14, 14]
Conv2d(64, 128, 4, 2, 1) → BN → LeakyReLU(0.2) → [B, 128, 7, 7]
Conv2d(128, 256, 3, 1, 1) → BN → LeakyReLU(0.2) → [B, 256, 7, 7]
Flatten → Linear(256*7*7, 1) → Sigmoid
```

- **Agent stops here. Explain:**
  - Why LeakyReLU (not ReLU) for discriminator: allows small gradients for negative activations
  - Standard ReLU causes "dying neurons" in D — LeakyReLU prevents it
  - What alpha=0.2 means in LeakyReLU: f(x) = x if x>0 else 0.2*x
  - Why no BN in first D layer: standard DCGAN practice (BN after first layer can hurt)
  - Sigmoid at output: probability of being real
- Wait for confirmation

STOP 6 — Weight Initialization (DCGAN Standard)

```python
def weights_init(m):
    classname = m.__class__.__name__
    if 'Conv' in classname:
        nn.init.normal_(m.weight, 0.0, 0.02)
    elif 'BatchNorm' in classname:
        nn.init.normal_(m.weight, 1.0, 0.02)
        nn.init.constant_(m.bias, 0)
```

- **Agent stops here. Explain:**
  - Why specific initialization for GANs: random initialization from N(0, 0.02) — small scale
  - Why BN weight initialized to 1.0 and bias to 0: maintain signal scale at start
  - Why GAN initialization is more critical than regular networks: training stability
  - The original DCGAN paper (Radford et al., 2015) established these as default practices
- Wait for confirmation

STOP 7 — Label Smoothing

- Instead of real=1, use real=0.9 (soft labels)
- Instead of fake=0, keep fake=0 (or 0.1 for double-sided smoothing)
- **Agent stops here. Explain:**
  - Why label smoothing helps D: prevents D from becoming overconfident
  - Overconfident D → very small gradients for G → G can't improve
  - Label smoothing keeps D gradients healthy throughout training
  - One-sided (only smooth real labels) is generally better for GANs
- Wait for confirmation

STOP 8 — Adversarial Training Loop

```python
for epoch in range(num_epochs):
    for real_batch in dataloader:
        # === Train Discriminator ===
        D.zero_grad()
        real_pred = D(real_batch)
        loss_D_real = criterion(real_pred, real_labels * 0.9)  # smooth

        z = torch.randn(batch_size, 100)
        fake = G(z).detach()  # detach: don't backprop into G yet
        fake_pred = D(fake)
        loss_D_fake = criterion(fake_pred, fake_labels)
        loss_D = loss_D_real + loss_D_fake
        loss_D.backward()
        optim_D.step()

        # === Train Generator ===
        G.zero_grad()
        z = torch.randn(batch_size, 100)
        fake = G(z)
        fake_pred = D(fake)
        loss_G = criterion(fake_pred, real_labels)  # fool D
        loss_G.backward()
        optim_G.step()
```

- **Agent stops here. Explain:**
  - Why `.detach()` when training D: don't want gradients flowing into G during D's step
  - Why we generate new z for G step (not reuse fake): different noise = more diverse gradients
  - The alternating training: one D step then one G step per batch
  - Why we use Adam with lr=0.0002, betas=(0.5, 0.999) — DCGAN paper recommendation
  - What `betas=(0.5, 0.999)` means: momentum decay rate — lower first moment for more volatile GAN training
- Wait for confirmation

STOP 9 — Training Monitoring

Every 5 epochs:

- Save a fixed grid of 64 generated images (using a FIXED z noise vector — same z each time)
- Plot D_loss, G_loss per epoch
- Monitor: D_loss ≈ 0.7 and G_loss ≈ 0.7 is healthy (D can't do better than random)
- **Agent stops here. Explain:**
  - Why fixed z for monitoring: can see training progression on the same latent points
  - The Nash equilibrium signal in losses: both converge near log(2) ≈ 0.693
  - What D_loss → 0 means: D dominates, G produces garbage
  - What G_loss → 0 means: G dominates, D is fooled completely (often mode collapse)
  - What oscillating losses mean: adversarial instability
- Wait for confirmation

Notebook 03 — Evaluation (`03_evaluation.ipynb`)

STOP 10 — Mode Coverage Analysis

- Generate 1000 images
- Use a pretrained MNIST classifier to assign labels to generated Fashion-MNIST images
- Check: does G generate all 10 categories, or is it biased toward some?
- **Agent stops here. Explain:**
  - What mode coverage measures: diversity of generated samples
  - How to detect mode collapse quantitatively: entropy of predicted class distribution
  - Perfectly diverse generation: uniform distribution across 10 classes
  - Mode collapse: one or two classes dominate
- Wait for confirmation

STOP 11 — FID Score Concept

- Compute FID (Fréchet Inception Distance) manually:
  - Extract features from a pretrained classifier for real images and generated images
  - Compute mean and covariance for each
  - $FID = ||\mu_real - \mu_fake||^2 + Tr(\Sigma_real + \Sigma_fake - 2\sqrt{(\Sigma_real * \Sigma_fake)})$
- **Agent stops here. Explain:**
  - What FID measures: distance between real and generated distributions in feature space
  - Lower FID = better (0 = identical distributions)
  - Why FID is preferred over just visual inspection (objective, not subjective)
  - The Fréchet distance: how similar are two multivariate Gaussians?
  - FID requires many samples (minimum 10k for reliable estimate)
- Wait for confirmation

STOP 12 — Latent Space Interpolation

- Pick two random z1, z2
- Generate 10 intermediate z: `z = slerp(t, z1, z2)` (spherical interpolation)
- Show as a row of 10 images
- **Agent stops here. Explain:**
  - What spherical interpolation (slerp) is vs linear: follows the surface of the hypersphere
  - Why slerp is better than linear for Gaussian noise: linear interpolation moves through the center (low probability region)
  - How to implement slerp: `slerp(t, v0, v1) = sin((1-t)*Ω)*v0/sin(Ω) + sin(t*Ω)*v1/sin(Ω)` where Ω = arccos(v0·v1)
- Wait for confirmation

STOP 13 — Save Models

- Save `generator.state_dict()` and `discriminator.state_dict()` separately
- Save fixed_z tensor (for reproducible demo generation)
- Write `generate(n)` → grid of n images
- **Agent stops here. Explain:**
  - Why we save both G and D (D needed if we want to continue training)
  - For pure inference: only G is needed
  - Why we save fixed_z: for app demo, always shows the same generated samples
  - How to handle the Tanh output → rescale to [0,1] for display: `(img + 1) / 2`
- Wait for confirmation

`dashboard_core.py`

Functions:

- `load_generator()` → generator model
- `generate_images(n=64)` → PIL grid image
- `get_training_curves()` → G_loss, D_loss arrays
- `get_epoch_grids()` → list of saved epoch grid paths
- `get_mode_coverage()` → dict of class distribution in generated images

`app.py` — Streamlit (~80 lines)

Sections:

1. "Generate" button → show 8×8 grid of new generated fashion images
2. Noise slider (random seed) → explore different z values
3. Tab 1: G_loss and D_loss training curves
4. Tab 2: Epoch-by-epoch grid progression (training progress)
5. Tab 3: Mode coverage bar chart (diversity of generated classes)

Key Concepts Covered

- GAN minimax game: min_G max_D formulation
- Non-saturating generator loss (flip labels trick)
- ConvTranspose2d for upsampling (generator)
- LeakyReLU for discriminator
- DCGAN weight initialization (N(0, 0.02))
- Label smoothing for training stability
- .detach() in adversarial training loop
- Mode collapse detection and monitoring
- FID score concept and computation
- Spherical interpolation (slerp) in latent space
- Training equilibrium signal (both losses ≈ 0.693)